

Wikidown Documentation

Generated 2026-09-29

Table of Contents

- [Home](#).....3
- [Getting Started](#).....4
 - [Format](#)..... 5
 - [Visual Studio Extension](#)..... 9
 - [Updating](#)..... 11
 - [Publishing to GitHub Pages](#)..... 13
 - [Publishing to GitLab Pages](#)..... 16
 - [Customizing the Theme](#)..... 17
 - [Kanban](#)..... 19
- [CLI](#)..... 21
- [Editor](#)..... 25
- [Agents](#)..... 27
- [Handy Prompts](#)..... 29
- [Release Notes](#)..... 30
- [Testing](#)..... 35
 - [Browser Test Plan](#)..... 36
- [Meta](#)..... 41
 - [Vibing Phase Recap](#)..... 42
 - [Release Quirks](#)..... 46

Home

Wikidown is a structured markdown wiki that lives in `/docs` of any git repo — a C# CLI (the surface AI agents use too), a Blazor WASM browser editor, and a marketing site, all built on the same page model. This wiki (the one you're reading) is itself a Wikidown wiki, dogfooding the format and tools it documents — including this site, which is the wiki published with `wikidown export -html`.

Where to go

- [Getting Started](#) — what a Wikidown wiki looks like on disk, the on-disk format spec, and how a wiki stays current
- [CLI](#) — the `wikidown dotnet` tool: `list` / `read` / `write` / `move` / `reorder` / `search` / `check-links` / `backfill-breadcrumbs`
- [Editor](#) — the browser-based Blazor WASM editor that commits straight to your repo
- [Agents](#) — drop-in Claude Code and GitHub Copilot configs for maintaining a Wikidown wiki with an AI agent
- [Handy Prompts](#) — example prompts to give your AI agent for everyday wiki upkeep
- [Release Notes](#) — what changed in each release and where to get it

Getting Started

Welcome to Wikidown. This folder is itself a Wikidown-format wiki.

Format quick reference

- Page file on disk: `Getting-Started.md`; rendered title: `Getting Started`.
- Subpages go in a folder with the same base name next to the page file.
- Ordering lives in `.order` — one page base-name per line.
- Body links are relative file paths (with the `.md` extension), not absolute title paths — see [Format](#) for the full rule.

Next

- [Format](#) — full on-disk format reference
- [Visual Studio Extension](#) — add a Wikidown project in VS 2022+
- [Updating](#) — how downstream repos pick up new CLI releases, agent config changes, and VS extension updates
- [Publishing to GitHub Pages](#) — publish the wiki as a static site with `wikidown pages` and GitHub's built-in Jekyll, or anywhere via `wikidown export-html`
- [Publishing to GitLab Pages](#) — one CI job running `wikidown export-html`, no Jekyll or Ruby
- [Customizing the Theme](#) — swap the wiki page layout or replace the root `index.html` with a hand-authored landing page
- [Kanban](#) — keep a simple To Do / Doing / Done board in your wiki with `wikidown kanban init`
- [CLI](#) — `wikidown dotnet` tool
- [Editor](#) — Blazor WASM browser editor
- [Agents](#) — drop-in Claude + Copilot configs
- [Handy Prompts](#) — example prompts for everyday wiki work: adding pages, editing content, reordering, and committing changes

Format Specification

Wikidown's on-disk format is deliberately minimal: markdown pages, folder-based hierarchy, and `.order` navigation files. The goal of this specification is to ensure that a repository's documentation is equally readable by humans browsing the file system, AI agents using the CLI, and web-based renderers.

By enforcing these rules, Wikidown prevents the link rot and structural drift that typically plagues flat-file documentation.

1. Page Files and Titles

Every page in the wiki is a standard Markdown file (`.md`). The title of the page is derived directly from its filename by replacing hyphens with spaces.

- **File on disk:** `Release-Notes.md`
- **Rendered title:** `Release Notes`

The reverse is also true: when creating a page titled "Getting Started", the file must be named `Getting-Started.md`.

2. Subpages and Hierarchy

Wikidown supports infinite nesting of pages. To create subpages for a given parent page, you must create a folder with the exact same base name as the parent page's file, located in the same directory.

- **Parent page:** `/docs/Architecture.md`
- **Subpage folder:** `/docs/Architecture/`
- **Child page:** `/docs/Architecture/Data-Model.md`

This sibling-folder structure ensures that deleting or moving a parent page can easily include all of its children.

Every subpage folder's parent page should exist and should **link every child in its body** — see § Index pages below. `WikiRepository.Write` will happily create `/Architecture/Data-Model` even if `/Architecture` doesn't exist yet, which silently orphans the whole subtree: `wikidown list / wikidown search / wikidown check-links` normal link scan all walk the wiki by descending from already-discovered pages, so a page whose parent was never created is invisible to all of them. `check-links` catches this specific case — see below.

3. Ordering

Alphabetical sorting is rarely the correct way to read documentation. Wikidown uses `.order` files to explicitly define the navigation hierarchy.

Every folder in the wiki (including the root `/docs` folder) should contain an `.order` file. This is a plain text file that lists the base names of the pages in that folder, one per line, from top to bottom.

Example `.order` file:

```
Getting-Started
Architecture
API-Reference
```

If a page exists in the folder but is not listed in the `.order` file, it is typically appended to the end of the list alphabetically by the renderer.

`.order` controls navigation-widget ordering only — it has no effect on GitHub's raw file view, so it's never a substitute for real body links. See § Index pages.

4. Internal Links

Internal links between wiki pages must be **relative file paths**, not absolute title paths. GitHub resolves an absolute path like `/Architecture/Data-Model` against the *repository* root, not the wiki root, so a link written that way 404s when the page is viewed directly on github.com. A relative path resolves correctly both on GitHub and in Wikidown-aware renderers.

Write the link relative to the **linking page's own folder**, adjusted for depth, and include the `.md` extension:

- **Correct (sibling page):** `[Read the Data Model](Data-Model.md)`
- **Correct (page in a different folder):** `[Read the Data Model](../Architecture/Data-Model.md)`
- **Incorrect:** `[Read the Data Model](/Architecture/Data-Model)`

Images and other repo assets follow the same rule, e.g. `![map](../attachments/map.png)`.

Run `wikidown check-links` (see [CLI](#)) to walk every page and verify that relative links and image references resolve to real files; by default it also flags any absolute title-path links left in page bodies, and audits that every folder has a linked index page (§ Index pages).

This rule only applies to links **inside page bodies**. Addressing a page through the CLI — e.g. `wikidown read --path /Getting-Started/Format` — still uses the absolute title path, since that's a command argument rather than a rendered link.

Fixing check-links failures

`check-links` prints one line per issue:

```
page:line -> target (reason)
```

What to do depends on the (reason):

- **(absolute title-path link (404s on GitHub))** — the link uses a `/Title/Path` form instead of a relative `.md` path. Rewrite it relative to the *linking page's own folder*, with the `.md` extension. For example, if `/Foo.md` (at the wiki root) contains `[Bar](/Bar)` and `Bar.md` is also at the wiki root, change it to `[Bar](Bar.md)`. If `/Sub/Foo.md` links to root-level `/Bar`, it becomes `[Bar](../Bar.md)`. Count the folder hops between the linking page and the target page to get the right number of `../`.
- **(upper/lower case differs from the file on disk (404s once published))** — the target exists, but not with that capitalisation: `[Format](format.md)` against a file named `Format.md`. Windows and macOS open it anyway; the Linux hosts a wiki is published to do not. Fix the link to match the file name exactly — or, if the file is the one that's wrong, rename it with `wikidown move --from Format --to format` (case-only renames work on every platform).
- **(broken link)** — a relative link/image that doesn't resolve to a real file. This is either a typo in the relative path or a stale link left over from a page that moved or was deleted. Open the linking page's folder and confirm the target filename, hop count, and `.md/.attachments` spelling; fix the path to match the real file. If the target page genuinely no longer exists, either remove the link or point it at wherever that content now lives.
- **A link that broke because a page moved** — `wikidown move` (see [CLI](#)) automatically rewrites inbound links and the moved page's own relative links when they change a page's path or folder depth. So a `check-links` failure pointing at a page that was clearly moved usually means one of two things: the move happened before that link-rewriting behavior shipped, or the link was added by hand (e.g. typed into a body) after the move rather than being created through `wikidown move`. Either way, fix it the same way as a broken relative link above — retarget it at the page's current path and depth.
- **(no index page <Folder>.md)** — a subpage folder exists but its sibling parent page is missing. Create it (`wikidown new --path /Folder`), then link every child from its body — see § Index pages.
- **(not linked from parent)** — the parent page exists but its body never links this child, so a reader browsing rendered markdown (or the raw file on GitHub, where `.order` means nothing) has no way to reach it. Add a relative link to the child in the parent's body — see § Index pages.

After editing, re-run `wikidown check-links` to confirm the line is gone.

5. Index Pages

A folder's parent page is that folder's **index** — the entry point a reader lands on before descending into its children. Two invariants keep that entry point real rather than aspirational, both audited by `wikidown check-links` (pass `--no-index-check` to skip this pass):

- **The index page must exist.** `/Architecture/Data-Model` can be created without `/Architecture` ever existing — nothing in `WikiRepository.Write` requires the parent first. When that happens the whole subtree becomes invisible to every wiki-model-based tool (`wikidown list`, `wikidown search`, and `check-links`' own link scan), since they all discover pages by descending from an already-discovered parent. `check-links` finds these orphans by walking the raw filesystem instead, specifically because it can't rely on the page model to see them.
- **The index page must link every child.** `.order` decides navigation order in Wikidown-aware UIs, but it's invisible on GitHub's raw file view — the only way a reader following links on `github.com` can reach a child page is a real link in the parent's body. `check-links` verifies every `*.md` file directly inside a folder is the resolved target of at least one link (relative or absolute) in that folder's parent page.

A page's own [breadcrumb](#) links upward to its ancestors, but that's a different direction from this check: the breadcrumb doesn't help a reader on the *parent* page discover its children, and it doesn't verify the ancestor pages it links to actually exist — a folder missing its index page still gets a breadcrumb pointing at a `.md` file that isn't there, which is exactly the case index-page auditing is meant to catch.

6. Breadcrumb Navigation

Every page gets a one-line breadcrumb trail auto-injected as its **first line**, always leading back to `/Home` when the wiki has one:

```
[Home](../Home.md) / [Encounters](../Encounters.md) / The Sky Hunters <!--  
wikidown:breadcrumb -->
```

This exists because GitHub's own file-path breadcrumb (shown above the raw file view) reflects the *repository* path — `docs / Encounters / The-Sky-Hunters.md` — not the wiki's page hierarchy or titles. Wikidown's breadcrumb is built purely from the page's own title chain instead: `Home` (if it exists) leads, then each further ancestor is a link (relative, per the convention above), and the current page's title is the final, unlinked segment — the same shape GitHub uses, just scoped to the wiki rather than the whole repo.

Key behavior:

- **Automatic.** `wikidown write` (and `new`, which writes through the same path) inject or refresh the breadcrumb on every save — there's nothing to opt into or maintain by hand. Write whatever body content you want; the first line is managed for you.
- **Idempotent.** The line carries an HTML comment marker (`<!-- wikidown:breadcrumb -->`) that's invisible when rendered. On every write, whatever was on line one gets discarded if it carries that marker, then a fresh one is computed from the page's *current* path — so re-saving a page never accumulates duplicate breadcrumbs, and a round-tripped `read` → `edit` → `write` doesn't need to preserve the line itself.
- **Always leads back to /Home, when the wiki has one.** `wikidown init` seeds a `/Home` page as the wiki's entry point; every other page's breadcrumb — including top-level pages, which otherwise have no ancestors — leads with a link to it, so there's always a way back. A wiki with no `/Home` page (this repo's own `/docs` predates that convention) never gets a fabricated link to a page that doesn't exist: top-level pages simply have no breadcrumb, and nested pages' breadcrumbs start from their nearest real ancestor, same as before `/Home` was introduced. `/Home` itself never links to itself.
- **Moves regenerate it, not just patch it.** `wikidown move` fully regenerates the breadcrumb for the moved page and every moved descendant, rather than trying to edit the existing links in place — a move can change *which* ancestors a page has, not just how many `../` hops reach them, so a patch-in-place approach can leave a structurally wrong (if still technically valid) breadcrumb behind.
- **Checked like any other link.** Breadcrumb links are ordinary relative markdown links, so `check-links` validates them the same way it validates everything else in the body — but only that they *resolve*.

Whether the ancestor page they point at *should* exist and is properly indexed is § Index Pages' job, not this one.

- **Wikis that predate /Home, or predate breadcrumbs entirely**, won't have this line until each page is re-saved. Run `wikidown backfill-breadcrumbs` once to catch every page up in one pass — see [CLI](#).

7. Markdown Dialect

Wikidown relies on standard CommonMark. There are no proprietary macros or shortcodes required to render the core text. An MVP renderer only needs a standard markdown parser plus the filename↔title mapping logic described above.

Visual Studio Extension

The **Wikidown** Visual Studio extension surfaces your wiki's docs/ folder directly in Solution Explorer — without it participating in the build.

Install

1. Open **Extensions → Manage Extensions** in Visual Studio 2022 or later.
2. Search for **Wikidown** in the Marketplace tab and click **Install**.
3. Restart Visual Studio when prompted.

Alternatively, download Wikidown.Vs.vsix directly from the [Visual Studio Marketplace](#) and double-click to install.

Add a Wikidown project to your solution

1. Right-click the solution node in **Solution Explorer** and choose **Add → New Project**.
2. Search for **Wikidown** (or filter by project type **Wikidown**).
3. Select **Wikidown Wiki** and click **Next**.
4. Give the project a name (default: wiki) and pick any folder you like — the location field defaults sensibly — then click **Create**.

When the project is created, the extension looks for an existing docs/ folder near the project file (checking the project folder and up to two parent folders). If it finds one, the new project points at it; if not, it creates a docs/ folder next to the project file seeded with a starter Home .md page. The wiki folder appears immediately under the new project node in Solution Explorer.

Configure the wiki root

By default the project looks for a docs/ folder next to the .wikidownproj file. Edit the file to point to a different location:

```
<WikidownProject>
<WikiRoot>my-wiki</WikiRoot> <!-- relative to the .wikidownproj file -->
</WikidownProject>
```

Features

- Presents the wiki the way Azure DevOps does: nodes show page **titles** (dashes render as spaces, no .md extension), a page with a same-named subpage folder is a single expandable node, .order files are hidden, and pages sort by .order (listed entries first, then alphabetical).
- Double-clicking a page opens it in VS's built-in markdown editor.
- Right-click the project, a folder, or a page for the standard context menus, including the **Git** submenu, plus wiki-native commands:
 - **Add Page...** prompts for a name, creates the page, and records it in .order.
 - **Add Folder...** creates a folder plus a same-named page beside it, following the CLI's subpage convention.
 - **Add Existing Pages...** copies .md files in and records them.
 - **Move Page Up / Move Page Down** reorder the page within its folder's .order.
 - **Edit Page Order** opens the folder's .order in the editor (creating it from the current display order if missing).
 - **Delete Page** (a page and, for pages with children, its subpages) and **Delete Folder** confirm and then delete — your git history is the undo.
 - **Export to PDF...** — right-click the project root, a folder, or a page and export it to a single linked PDF: the selected node plus all of its descendants (the project root exports the whole wiki). Prompts

for a save location, then generates the PDF the same way `wikidown export-pdf` does (cover page, table of contents, per-page bookmarks/outline, embedded fonts) — if the `wikidown` CLI is installed as a global dotnet tool it is used (so `dotnet tool update -g Wikidown.Cli` keeps the export current); otherwise the extension falls back to a copy bundled inside the VSIX, so no separate install step is required. Offers to open the finished PDF when done.

- Everything stays consistent with the `wikidown` CLI, which remains fully usable side by side — the tree refreshes live when it changes files or reorders `.order`.
- The project node is **display-only** — it never appears in Build, Rebuild, or Clean, and adds no compile items to the solution.

Add an existing project to a `.sln` manually

If you prefer to hand-edit the solution file, use the Wikidown project type GUID:

```
Project("{6a9c3f4b-d5e8-4f0a-b1c2-345678901bcd}") = "wiki",  
"wiki.wikidownproj", "{<new-guid>}"  
EndProject
```

Related

- [CLI](#) — `wikidown` dotnet tool for editing from the terminal, a CI pipeline, or an AI agent.
- [Agents](#) — drop-in Claude Code and GitHub Copilot configs that drive the CLI.
- [Editor](#) — browser-based Blazor editor that commits directly to your repo.

Updating

How a downstream repo picks up new Wikidown releases — the CLI tool version, agent config files, and the VS extension each update differently.

CLI (NuGet global tool)

Wikidown.Cli is published to NuGet as a global .NET tool. Update it with:

```
dotnet tool update -g Wikidown.Cli
```

A push to main doesn't always mean a new version is available. The [release.yml](#) workflow runs on every push to main that touches `Directory.Build.props`, `src/Wikidown.Cli/**`, `src/Wikidown.Core/**`, `assets/**`, or the workflow file itself, and it packs + pushes to NuGet with `--skip-duplicate`. The package version comes from `<VersionPrefix>` in `Directory.Build.props` — and that value is **not** auto-incremented per push. `--skip-duplicate` means a push that doesn't bump `VersionPrefix` re-packs and tries to push the *same* version, which NuGet rejects as a duplicate and the step just no-ops. So a plain code-fix push only becomes something `dotnet tool update` can see once a maintainer bumps `VersionPrefix` in the same push (or a later one).

If `dotnet tool update` reports you're already on the latest version but you expected a fix, check whether `VersionPrefix` has actually moved since your last update — the fix may be merged but not yet released.

Self-contained CLI binary (no-.NET installs)

If wikidown was installed by the `wikidown.org` install script on a machine without .NET, it's a self-contained binary in `~/.wikidown/bin` (`%USERPROFILE%\\.wikidown\bin` on Windows) — `dotnet tool update` doesn't apply to it. Update by re-running the script; it always fetches the newest native release:

```
curl -fsSL https://wikidown.org/install.sh | sh
```

```
irm https://wikidown.org/install.ps1 | iex
```

Native binaries are built by [cli-native.yml](#) when a maintainer pushes a `cli-v*` tag — a **separate release track** from NuGet, so the newest NuGet version and the newest native binary aren't necessarily the same; if the script reinstalls and a fix still isn't there, check whether a `cli-v*` tag has been cut since the fix merged.

Agent configs (.claude/, .github/, CLAUDE.md)

These files are copied into a downstream repo once, by `wikidown init` (see [CLI](#) and [Agents](#)) — they don't auto-update. To pick up changes to the shipped configs (wording tweaks, new instructions, new tool wiring), re-run `init` with `--force`:

```
wikidown init --agents all --force
```

`--force` is required because `init` otherwise skips any file that already exists in the target repo. Review the diff before committing — `--force` overwrites local edits to those files too.

VS extension (VSIX)

The Visual Studio extension auto-updates through the **Visual Studio Marketplace's** normal extension update mechanism once a new version is published. Publishing is triggered by pushing a `vsix-v*` tag (see [vsix.yml](#)), which builds the VSIX, runs `VsixPublisher.exe` against the Marketplace, and attaches the `.vsix` to the GitHub Release. No action is needed downstream beyond letting Visual Studio check for extension updates as usual — see [Visual Studio Extension](#).

Summary

Component	Trigger to publish	How consumers update
Wikidown.Cli (NuGet)	Push to main touching Core/CLI/Directory.Build.props, and a VersionPrefix bump	dotnet tool update -g Wikidown.Cli
Self-contained CLI binary	Push of a cli-v* tag	Re-run the wikidown.org install script
Agent configs	N/A — copied at init time	wikidown init --agents all -force
VS extension (VSIX)	Push of a vsix-v* tag	Automatic via VS Marketplace update check

Publishing to GitHub Pages

GitHub Pages can publish a Wikidown wiki straight from the repo using Jekyll, its built-in static-site generator — no build pipeline, and nothing to install: GitHub's servers run Jekyll, your machine never does. `wikidown` pages scaffolds everything Jekyll needs into the wiki root, plus a starter theme with a left-navigation tree that follows your `.order` files.

Not on GitHub, or want to preview locally? `wikidown export-html` renders the **same theme** in .NET with no Jekyll or Ruby involved — see [Any other host: export-html](#) below, and [Publishing to GitLab Pages](#) for the GitLab walkthrough. That's how [wikidown.org](#) itself is published: it *is* this wiki, exported by CI on every change.

Quick start

```
wikidown pages # run from the repo root; --root docs is the default
git add docs
git commit -m "Publish wiki with GitHub Pages"
git push
```

Then in GitHub: **Settings** → **Pages** → **Source** "Deploy from a branch", branch `main`, folder `/docs`. The wiki is live at `https://<owner>.github.io/<repo>/` a minute later.

Options:

- `--title T` — site title; defaults to the repo folder name.
- `--force` — overwrite theme files you've edited. **Careful:** this also replaces a customized root `index.html` with the stock redirect.

What it scaffolds

Everything lands inside the wiki root:

File	Purpose
<code>_config.yml</code>	Jekyll config: GFM markdown, the three GitHub-bundled plugins (<code>jekyll-relative-links</code> , <code>jekyll-titles-from-headings</code> , <code>jekyll-default-layout</code>), <code>include: [.attachments]</code> so images work (Jekyll skips dot-folders by default), default layout <code>wikidown</code> . Also the home of Wikidown's own site settings: <code>repository_url</code> (GitHub link in the top bar), <code>favicon</code> , and <code>wikidown.exclude_from_site</code> (below).
<code>_data/navigation.yml</code>	The nav tree, generated from <code>.order</code> . Regenerated automatically by the CLI on every write/move/delete/reorder once it exists — never edit by hand.
<code>_layouts/wikidown.html</code> , <code>_includes/nav-tree.html</code>	The starter theme: top bar, collapsible left nav (active page highlighted, ancestors expanded), content column, footer. Responsive — the nav becomes a slide-in drawer on narrow screens.
<code>assets/wikidown.css</code>	Styling, same palette as <code>wikidown.org</code> . Edit freely; pages never overwrites it without <code>--force</code> .

index.html	Redirects the site root to /Home.html (or the first top-level page if there is no Home). Replace it with a hand-authored landing page if you want one — wikidown.org does exactly that.
------------	---

Customizing any of these — the wiki page layout or the root index.html — is its own topic: see [Customizing the Theme](#).

Excluding pages from the published site

Internal notes can stay in the wiki without shipping to the site. List subtrees under `wikidown.exclude_from_site` in `_config.yml`:

```
wikidown:
exclude_from_site:
- /Meta
- /Testing
```

This is a **publishing** concern only: excluded pages remain first-class for the CLI, editor, and `check-links` — they just produce no `.html` and no nav entry. `export-html` honors it fully; `_data/navigation.yml` is generated without them too, so the Jekyll path hides them from the nav — but GitHub's builder still *emits* their pages, so when using built-in Jekyll also list them under the top-level `exclude`: key to keep the files out entirely.

One thing the tools can't do for you: body links pointing *into* an excluded subtree will 404 on the published site. `check-links` still validates them against the repo (where the pages exist), so trim such links from published pages yourself.

Why it works with the Wikidown format

- Relative `.md` links (including the auto breadcrumb) are rewritten to the generated `.html` URLs by `jekyll-relative-links`.
- Page titles come from the first `#` Heading, so pages need no YAML front matter.
- `.order` files are dotfiles, so Jekyll ignores them; the nav tree reads them via `_data/navigation.yml` instead, because GitHub's Pages builder can't run custom plugins.
- `wikidown check-links` ignores `_`-prefixed folders and folders with no markdown, so the scaffolded files don't trip the index-page audit.

Gotchas

- **Liquid in page bodies.** Jekyll processes `{ { }}` and `{% %}` everywhere, including code blocks. Wrap such content in `{% raw %} ... {% endraw %}`. (`export-html` does *not* run Liquid over page bodies, so it's unaffected.)
- **Project sites live under `<repo>/`.** The theme uses `relative_url` everywhere so this just works; if you serve from a custom domain, set `baseurl: ""` in `_config.yml`.
- **Don't `.nojekyll`** when using the built-in Jekyll path — that file disables Jekyll and would serve raw `.md` files. (An `export-html`-based deploy *should* write `.nojekyll`, since the output is already final HTML.)
- **Keep `_data/navigation.yml` committed.** If it's missing the layout falls back to a flat alphabetical page list.

Any other host: `export-html`

```
wikidown export-html --output public
```

Renders every page through the same `_layouts/wikidown.html`, `_includes/nav-tree.html`, and `assets/wikidown.css` that GitHub's Jekyll would use — but in-process, with [Markdig](#) for markdown and

[Fluid](#) for Liquid. The output folder is a complete static site: one `.html` per page, the root `index.html`, `assets/`, `.attachments/`, and **every other static file in the wiki root** (images, install scripts, favicons, extra stylesheets) copied through verbatim — wiki sources (`.md`, `.order`), `_`-prefixed Jekyll machinery, other dot-files, and Gemfiles stay out. Nothing to install beyond the CLI.

- Works **with or without** having run `wikidown` pages. If the wiki root has theme files, they're used (so your customizations apply); anything missing falls back to the built-in copy. `_config.yml`'s `title`, `description`, `repository_url`, `baseurl`, `favicon`, and `wikidown.exclude_from_site` are honored.
- `--base-url /prefix` — prefix for theme links (stylesheet, nav, redirect) when the site isn't served from the domain root, e.g. GitLab project sites at `https://<group>.gitlab.io/<project>/`. Overrides `baseurl` in `_config.yml`. Links inside page bodies are relative and never need it.
- `--title T` — overrides the site title.
- `--clean` — delete the output folder first, so removed pages don't linger.

Links, titles, and the nav tree match the Jekyll output: relative `.md/.md#anchor` links become `.html`, the title is the first `#` Heading, and the sidebar is built from `.order` (live, not from `_data/navigation.yml`, so it's always current).

Local preview is just the export plus any static file server, e.g. `dotnet serve -d public` (dotnet tool `install -g dotnet-serve`) or `npx serve public`. Theme authors can iterate on `_layouts/assets` this way before pushing to GitHub.

Theme files are written in Jekyll's Liquid dialect; `export -html` translates the one construct that differs (`{% include file a=b %}` / `include .a`) on the fly, so a single theme serves both paths. Stick to standard Liquid filters and tags — Jekyll-only extras like `where_exp` aren't available in the .NET renderer.

GitLab Pages

Covered on its own page: [Publishing to GitLab Pages](#) — a one-job `.gitlab-ci.yml` that runs `export -html` in a .NET container. The same recipe works for any CI that deploys a static folder.

See [CLI](#) for the full command list, [Format](#) for the on-disk conventions the generated site relies on, and [Customizing the Theme](#) for swapping the wiki layout or the root `index.html`.

Publishing to GitLab Pages

GitLab Pages serves static sites, but unlike GitHub it doesn't run a site generator for you — it publishes whatever a CI job leaves in a `public/` artifact. That suits Wikidown fine: `wikidown export-html` renders the wiki to finished HTML with the same starter theme the GitHub path uses, in one CI job, with no Jekyll or Ruby anywhere (see [Publishing to GitHub Pages](#) for the GitHub story and the full `export-html` reference).

Steps

1. (Optional) Run `wikidown pages` and commit the result, if you want the theme files (`_layouts`, `assets`, `_config.yml`) in your repo to customize. `export-html` works without this — anything missing falls back to the built-in theme.
2. Add `.gitlab-ci.yml` at the repo root:

```
pages:
image: mcr.microsoft.com/dotnet/sdk:10.0
stage: deploy
script:
- dotnet tool install -g Wikidown.Cli
- export PATH="$PATH:$HOME/.dotnet/tools"
- wikidown export-html --output public --base-url "/$CI_PROJECT_NAME" --clean
artifacts:
paths:
- public
rules:
- if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
```

The job must be named `pages` and the artifact folder must be `public` — both are GitLab conventions.

3. Push. **Deploy → Pages** in the project shows the URL once the job finishes — `https://<group>.gitlab.io/<project>/` by default. Pages must be enabled on the instance (it is on [gitlab.com](#)).

--base-url

Project sites are served under `/<project>/`, and the theme's own links (stylesheet, nav, root redirect) need that prefix — `--base-url "/$CI_PROJECT_NAME"` supplies it. Drop the flag if the project uses a **custom domain** or a **group-level site** served from `/`. Links inside page bodies are relative and never need it.

Notes

- The CI container runs .NET, so nothing is installed on your machine — same as the GitHub path.
- `wikidown.exclude_from_site`, `favicon`, `title`, and the other `_config.yml` settings are honored — see [Publishing to GitHub Pages](#) for what each does.
- The same three-line script works for any CI that deploys a static folder: Azure Static Web Apps, Netlify, Cloudflare Pages, S3.
- Want a custom `_layouts/wikidown.html/assets/wikidown.css` or a hand-authored root `index.html` instead of the stock redirect? Same files, same rules as the GitHub path — see [Customizing the Theme](#).

See [CLI](#) for the full `export-html` option reference.

Customizing the Theme

wikidown pages scaffolds two independent templates. Both are plain files in the wiki root, and both are optional to edit — but they solve different problems and get replaced in different ways.

Template	Wraps	Purpose
<code>_layouts/wikidown.html + _includes/nav-tree.html + assets/wikidown.css</code>	Every generated wiki page (<code>Home.html</code> , <code>Guides.html</code> , ...)	Page chrome: top bar, the .order-driven left-nav sidebar, footer
<code>index.html</code>	Nothing — it's standalone	The site root; free to look nothing like the wiki

This page covers customizing both. For the scaffold itself and publishing mechanics, see [Publishing to GitHub Pages](#) and [Publishing to GitLab Pages](#).

The wiki page template

`_layouts/wikidown.html` is the Liquid layout every rendered page goes through: `<head>`, top bar, a slot for `{{ content }}` (the rendered markdown), and the sidebar. `_includes/nav-tree.html` renders that sidebar recursively from `site.data.navigation` — the tree built from your `.order` files — with active-page highlighting and collapsible sections. `assets/wikidown.css` is the styling.

To customize: edit those three files directly in the wiki root. Both GitHub's built-in Jekyll and wikidown `export-html` prefer wiki-root copies over their embedded defaults, so your edits apply either way. Nothing regenerates them once they exist — the only thing that overwrites them is `wikidown pages --force`.

Both files are standard Liquid. `export-html`'s Fluid renderer translates the one Jekyll-only construct it needs — `{% include file.html a=b %}` becomes `include.a` — so sticking to standard Liquid filters and tags keeps the layout working identically on both engines. Jekyll-only extras like `where_exp` aren't available in the .NET renderer.

The root `index.html`

Unlike every other generated page, `index.html` is **not** wrapped by `_layouts/wikidown.html` at all — the scaffolded file carries `layout: none` in its front matter, and both renderers treat it as fully standalone. That means it's free to look nothing like the rest of the wiki: no sidebar, its own nav bar, a hero section, feature cards — whatever a marketing homepage needs.

The default scaffolded version is a trivial redirect stub: a meta-refresh and a canonical link that send visitors straight to the wiki's home page, plus a visible fallback link for the no-JS/no-refresh case.

[wikidown.org's own `index.html`](#) is a real, working example of replacing it — hero section, feature cards, its own top nav that links into the wiki with a plain `href="Home.html"`. It's worth reading directly as the reference implementation to copy from: `docs/index.html` in this repo.

To customize: edit or replace `docs/index.html` after scaffolding. Both publishing paths read it straight off disk when present:

- `wikidown pages` only writes files that don't already exist, unless you pass `--force` — which clobbers a customized `index.html` back to the stock redirect (see the `--force` warning on [Publishing to GitHub Pages](#)).
- `wikidown export-html` layers wiki-root files over its embedded defaults — a wiki-root `index.html` always wins over the built-in one when present, and it doesn't need `wikidown pages` to have been run first.

The `{{HOME}}` / `{{TITLE}}` placeholders

The stock `index.html` contains two placeholders:

```
<meta http-equiv="refresh" content="0; url={{ '}}{{HOME}}' | relative_url }}">
<title>{{ site.title }}</title>
```

`{{TITLE}}` and `{{HOME}}` are **plain literal text**, not Liquid — they're substituted with a raw string replace over the file's text, done independently in two places:

- `wikidown pages` (`PagesCommand`) substitutes them once, at scaffold time, and writes the already-substituted text into the committed `index.html`. Real Jekyll — GitHub's own builder — never sees the placeholders at all; by the time it builds, they're already gone.
- `wikidown export-html` (`HtmlExporter`) substitutes them fresh on every export, so it keeps a hand-scaffolded `index.html` that still contains the placeholders self-updating even without rerunning `wikidown pages`.

`{{HOME}}` resolves to the wiki's `/Home` page if one exists, else the first top-level page, else `/Home.html` as a last resort — always with a `.html` suffix, since it's the concrete redirect target of a real generated file. (Links *within* rendered wiki pages — the sidebar, breadcrumbs — are different: those resolve the wiki's Home page to `/` instead of `/Home.html`, so a fully custom homepage that serves Home's content at the literal site root doesn't break them. The redirect stub itself doesn't need that, because `Home.html` always exists as its own file alongside `index.html`.)

Why the placeholder sits inside a Liquid string

Look closely at the source template (`src/Wikidown.Html/Theme/index.html`):

```
{{ '}}{{HOME}}' | relative_url }}
```

`{{HOME}}` is written inside a Liquid *string literal*, itself wrapped in the `relative_url` filter. The raw-text replace happens **before** any Liquid or Fluid parsing runs, so it substitutes the literal characters `{{HOME}}` inside the quotes — turning `'{{HOME}}'` into e.g. `'/Home.html'` — leaving well-formed Liquid that only gets evaluated afterward. That's how `baseurl`-prefixing still works on both engines: `relative_url` runs at each engine's own render time (real Jekyll's own filter on the GitHub Pages path, a custom-registered Fluid filter using `--base-url` for `export-html`) — the placeholder substitution only ever supplies the unprefixed link path, never the final href.

Practical takeaway if you're customizing `index.html`: reuse `{{ '}}{{HOME}}' | relative_url }}` somewhere in your markup if you want your "go to the docs" link to auto-track the wiki's actual home page instead of a URL you'd have to keep in sync by hand. It's optional, not required — `wikidown.org`'s own page skips it and hardcodes `href="/Home.html"` in its nav bar instead, which is fine too.

Previewing changes

`wikidown export-html --output public --clean` plus any static file server gives instant local preview of either kind of edit, with no Jekyll, Ruby, or GitHub Pages deploy required — see [Publishing to GitHub Pages](#) for the full workflow.

See [CLI](#) for the full option reference, and [Publishing to GitLab Pages](#) for a worked example of `export-html` in CI — the theme-customization mechanics here apply to any `export-html`-based host, not just GitHub.

Kanban

Keep a simple to-do board in your wiki: one dashboard page with To Do, Doing and Done sections, and one page per item.

Set up the board

```
wikidown kanban init
```

This creates a dashboard page, /Kanban, with three sub-pages: To Do, Doing and Done. The dashboard has one ## section per sub-page, and each heading links to it:

```
## [To Do](Kanban/To-Do.md)
## [Doing](Kanban/Doing.md)
## [Done](Kanban/Done.md)
```

Pass --path to use a different name, e.g. `wikidown kanban init --path /Home-Projects`.

There is nothing special about these pages. Each item is an ordinary sub-page of To Do, Doing or Done (for example /Kanban/To-Do/Buy-Paint), and every page follows the same rules as any other wiki page. You can edit any of them by hand.

Add an item

Create the item's page:

```
wikidown new --path Kanban/To-Do/Buy-Paint --title "Buy paint"
```

Then list it in two places: under "To Do" on the dashboard, and on the To Do page. A one-line block can be piped in from PowerShell:

```
"- [Buy paint](Kanban/To-Do/Buy-Paint.md)" | wikidown append --path Kanban -
-after "To Do" --stdin
"- [Buy paint](To-Do/Buy-Paint.md)" | wikidown append --path Kanban/To-Do --
stdin
```

Or save the line to a file and pass --file line.md instead of --stdin; that works in any shell. --after "To Do" finds the heading by its visible text, even though the heading is a link.

Move an item

When you start on an item, move its page to Doing:

```
wikidown move --from Kanban/To-Do/Buy-Paint --to Kanban/Doing/Buy-Paint
```

This updates both links to point at the new page, but it leaves the lines where they were. Finish by moving the lines yourself:

1. On the dashboard, remove the line from under "To Do" and add it under "Doing":

```
wikidown edit --path Kanban --old "- [Buy paint](Kanban/Doing/Buy-
Paint.md)" --delete
"- [Buy paint](Kanban/Doing/Buy-Paint.md)" | wikidown append --path
Kanban --after "Doing" --stdin
```

2. Remove the line from the To Do page and add it to the Doing page:

```
wikidown edit --path Kanban/To-Do --old "- [Buy paint](Doing/Buy-
Paint.md)" --delete
"- [Buy paint](Doing/Buy-Paint.md)" | wikidown append --path Kanban/Doing
--stdin
```

Do the same again to move it to Done.

Add a section

Want a "Waiting" column? Create its page and add a matching section to the dashboard:

```
wikidown new --path Kanban/Waiting  
"## [Waiting](Kanban/Waiting.md)" | wikidown append --path Kanban --stdin
```

Check the board

```
wikidown check-links
```

This reports broken links, and any item page that isn't listed on its section page — a sign that a line was missed after adding or moving an item. See the [CLI](#) page for every command.

CLI

The wikidown dotnet tool reads, writes, moves, reorders, searches, and link-checks pages in a Wikidown wiki. It keeps `.order` files, breadcrumb navigation, and (on move) links in sync automatically. It's also the surface AI agents use to maintain a wiki — see [Agents](#).

Install

```
dotnet tool install -g Wikidown.Cli
```

To update an existing install, see [Updating](#) — a plain push to main doesn't always mean a new NuGet version is available.

Quick install (no .NET required)

```
curl -fsSL https://wikidown.org/install.sh | sh
```

```
irm https://wikidown.org/install.ps1 | iex
```

Both scripts auto-detect: if dotnet is on PATH they run `dotnet tool install -g Wikidown.Cli` exactly as above; otherwise they download a self-contained single-file binary for your platform (win-x64/win-arm64/linux-x64/linux-arm64/osx-x64/osx-arm64) from this repo's GitHub Releases (tagged `cli-v*`), extract it, and add it to PATH. Either way nothing else needs installing — the binaries bundle their own .NET runtime.

The binaries aren't code-signed or notarized yet, so the scripts work around the resulting OS friction themselves: `install.ps1` runs `Unblock-File` on the downloaded exe so Windows SmartScreen doesn't flag it, and `install.sh` strips the `com.apple.quarantine` attribute on macOS so Gatekeeper doesn't block it. This is a known gap, not a bug — signing/notarization is planned but not done.

Either install path, the next step is the same: `wikidown init`.

Wiki root

The CLI defaults to `./docs`. Override with `--root <path>`:

```
wikidown list --root ./my-wiki
```

Page paths may be written with or without the leading slash — `--path Getting-Started/Format` and `--path /Getting-Started/Format` are the same page, and `.` is the wiki root (`--folder .`). Prefer the slash-less form under Git Bash on Windows, which rewrites arguments that start with `/` into `C:/Program Files/Git/...` before the CLI sees them. PowerShell and `cmd` pass them through untouched.

The CLI behaves the same on Windows, macOS, and Linux:

- **Text is UTF-8 everywhere**, including piped `--stdin` and redirected output (`wikidown read --path Home > home.md`). For multi-line text `--file` is identical in every shell. With `--stdin`, note that Windows PowerShell 5.1 re-encodes piped text as ASCII unless `$OutputEncoding` is set to UTF-8; PowerShell 7 already is.
- **Files keep their own line endings** — and their UTF-8 BOM, if they have one — when the CLI rewrites them. New files match the rest of the wiki, and a brand-new wiki uses the platform's default (CRLF on Windows). CRLF and LF both read fine, in pages and in `.order`; there is nothing to normalize.
- **New page names must be portable**. Characters Windows forbids (`\`, `:`, `*`, `?`, `"`, `<`, `>`, `|`, plus `#`), names ending in a dot or a space, and reserved device names (`CON`, `NUL`, `COM1`, ...) are refused on every platform, so a wiki authored on Linux still checks out on Windows.

Commands

- `init [--root docs] [--agents claude|copilot|all|none] [--force]` — seed an empty wiki with a /Home page and install the AI agent configs (Claude Code
 - GitHub Copilot) into the folder containing the wiki root. See [Agents](#). Re-run with `--force` to pick up updated agent configs in an existing repo — see [Updating](#).
- `list [--path /P]` — list children of a page (or root). `wikidown list`
- `walk [--path /P]` — list every page in the wiki, depth-first in `.order` order, one per line as `path<TAB>title`; with `--path`, only that page's descendants. The one-call way for an agent to orient itself before editing. `wikidown walk · wikidown walk --path /Getting-Started`
- `read --path /P [--section "<heading>"]` — print page markdown to stdout, or just one section of it. `--section` matches a heading case-insensitively, ignoring leading #s and whitespace, and prints that heading plus everything below it up to the next heading of the same or higher level (a ## section includes its ### children; headings inside code fences don't count). A miss lists the page's headings; if several headings match, the first is printed with a trailing note. A heading written as a link matches by its visible text, so `--section "To Do"` finds a heading that links the To Do page; the same goes for `--after` on `append` and `--section` on `write-section`. `wikidown read --path /Getting-Started · wikidown read --path /CLI --section "Wiki root"`
- `write --path /P [--file F | --stdin]` — overwrite a page. Auto-injects or refreshes the page's breadcrumb line — see [Format § Breadcrumb Navigation](#).
- `edit --path /P (--old T | --old-file F) (--new T | --new-file F | --stdin | --delete) [--all]` — replace an exact substring of a page in place, leaving the rest of the page untouched; the small-change alternative to `write`. The old text must match the page's raw markdown exactly (line endings are normalized to the file's, so CRLF vs LF never matters) and, without `--all`, exactly once — a miss or an ambiguous match is an error that says how many times the text matched. Use `--old-file/--new-file` (or `--stdin` for the replacement) for multi-line text. Never touches the breadcrumb line or `.order`, and never creates a page. Prints the changed line numbers with two lines of context. To remove text, pass `--delete` instead of a replacement — Windows PowerShell 5.1 drops an empty `--new ""` before the CLI ever sees it. `wikidown edit --path /Home --old "coming soon" --new "shipped" · wikidown edit --path /Home --old " (beta)" --delete`
- `write-section --path /P --section "<heading>" (--file F | --stdin) [--create]` — replace the body under one heading, keeping the rest of the page. The heading line is preserved verbatim and the body from a file or stdin replaces everything after it up to the next heading of the same or higher level — ### children inside a ## section included. Exactly one blank line is kept around the new body, so repeated writes don't stack blank lines. A miss lists the page's headings (`--create` appends a new ## <heading> instead), an ambiguous match is an error, and breadcrumb/.order/line endings are untouched. `wikidown write-section --path /CLI --section "Wiki root" --file root.md`
- `append --path /P [--after "<heading>"] (--file F | --stdin)` — add a block at the end of a page, or with `--after` at the end of that section's body, just before the next heading of the same or higher level. Exactly one blank line separates the block from existing content and the file ends with a single newline, so repeated appends never stack blank lines. A miss lists the page's headings, an ambiguous match is an error, and breadcrumb/.order/line endings are untouched. `wikidown append --path /Home --after "Open concerns" --file concern.md`
- `new --path /P [--title T] [--file F | --stdin]` — create a new page.
- `move --from /A --to /B [--dry-run]` — rename or move a page (subpages travel with it). Rewrites inbound links from every other page that pointed at the old path, rewrites the moved page's own relative links and images if the move changed its folder depth (e.g. moving /Encounters/Foo to /Adventures/Bar/Foo turns ../.attachments/x.png into ../../.attachments/x.png), and regenerates the breadcrumb for the moved page and every moved descendant. Reports a count and a per-link list of what changed. `--dry-run` previews the rewrite without touching any files.
- `kanban init [--path /Kanban]` — create a simple to-do board: a dashboard page (default /Kanban) with To Do, Doing and Done sub-pages and one ## section per sub-page, each heading linking its sub-page. Items are ordinary sub-pages added with `new` and moved with `move`. See [Kanban](#). `wikidown kanban init · wikidown kanban init --path /Home-Projects`

- `delete --path /P [--recursive]` — delete a page (and optionally its subpages).
- `reorder --folder /P --names a,b,c` — rewrite `.order` for a folder.
- `search --query <text>` — full-text search across page bodies.
- `check-links [--no-absolute-check] [--no-index-check]` — walk every page and validate that relative markdown links (`[x](../Foo/Bar.md)`) and image references (`![x](../attachments/pic.png)`) resolve to real files relative to the linking page's folder (links inside code blocks and inline code are examples and are skipped). The match is case-exact on every platform, so a link that only resolves because Windows or macOS ignores case is reported before it 404s on a case-sensitive host such as GitHub Pages. By default also:

- flags absolute title-path links in page bodies (`[x](/Foo/Bar)`), since GitHub resolves those against the repo root and they 404 when the wiki is browsed on `github.com` (`--no-absolute-check` to skip);
- audits that every subpage folder has an index page and that the index page links every child in its body, since `WikiRepository.Write` can create a grandchild page without its parent ever existing, silently orphaning the subtree from `wikidown list / wikidown search /` the rest of `check-links` itself (`--no-index-check` to skip). See [Format § Index Pages](#).

Prints one line per issue as `page:line -> target (reason)` (link issues) or `folder -> detail (reason)` (index issues), and exits non-zero if any issues are found, so it's CI-friendly. See [Format § Fixing check-links failures](#) for how to resolve each kind of reported issue.

- `backfill-breadcrumbs [--dry-run]` — one-time catch-up for a wiki that predates breadcrumb navigation (chunk 11): re-saves every page that has an ancestor but is missing its breadcrumb line, so it picks one up. Only needed once per existing wiki — `write`, `new`, and `move` all maintain breadcrumbs automatically going forward, so a wiki that's always been edited through this CLI never needs it. `--dry-run` lists which pages would change without writing anything. Prints a count.
- `export-pdf --output <path> [--from /Link/Path] [--title T] [--no-cover] [--no-toc] [--allow-html-skip]` — combine the whole wiki (or a subtree, with `--from`) into a single linked PDF:

- an optional cover page (title defaults to the repo's folder name, override with `--title`, skip with `--no-cover`);
- an optional in-document table of contents page with page numbers and clickable entries, indented to match the nav hierarchy (skip with `--no-toc`);
- one section per wiki page, with each page's own leading `# Title` heading placed at its nav depth, so the PDF's bookmark/outline panel mirrors the wiki's folder structure instead of listing every page flat;
- internal wiki links — relative `.md` links, legacy absolute title-paths, and same-page `#fragment` links to headings — become real in-PDF jumps, not dead hrefs;
- full markdown fidelity: headings, bold/italic, inline code, nested bullet/numbered lists, fenced code blocks, block quotes, pipe tables, and images (embedded when the relative path resolves).

A missing/broken image renders a visible placeholder and prints warning: `{page}: image not found: {target}` (mirroring `check-links`); the export still completes, but the command exits 1 if there were any warnings (0 otherwise). Raw HTML in a page's markdown (e.g. a `<div>`) is unsupported and fails the whole export by default — pass `--allow-html-skip` to instead render a `[unsupported HTML block omitted]` placeholder and continue. For example: `wikidown export-pdf --output wiki.pdf`.

PDF font resolution is cross-platform: an embedded `EmbeddedFontResolver` ships Atkinson Hyperlegible Next (body text) and Atkinson Hyperlegible Mono (code) TrueType fonts inside the `Wikidown.Pdf` assembly instead of resolving fonts from the host OS (SIL Open Font License 1.1). Characters those fonts lack (arrows, box drawing, Greek, Cyrillic) are set in the also-embedded `DejaVu Sans / DejaVu Sans Mono` instead of printing blank. `export-pdf` works identically on Windows, Linux, and macOS.

- `pages [--title T] [--force]` — scaffold everything GitHub Pages' built-in Jekyll needs to publish the wiki as a static site, straight from the repo with no build pipeline: `_config.yml`, a starter theme (top bar, collapsible left nav that follows `.order`, content column), `assets/wikidown.css`,

and a root `index.html` redirect. Nothing to install locally — GitHub runs Jekyll for you. The nav tree lives in `_data/navigation.yml`, which the CLI regenerates on every write/move/delete/reorder once it exists. `--title` sets the site title (defaults to the repo folder name); `--force` overwrites theme files you've edited. Commit the result, point **Settings → Pages** at the `/docs` folder, and the wiki is live. See [Publishing to GitHub Pages](#) for setup steps, what each scaffolded file does, and gotchas.

- `export-html --output <dir> [--base-url /prefix] [--title T] [--clean]` — render the wiki to a complete static site using the same theme pages scaffolds, in-process (Markdig + Fluid) with no Jekyll or Ruby. One `.html` per page, `index.html` redirect, `assets/` and `.attachments/` copied through, along with every other root static file (images, install scripts, favicons); relative `.md` links rewritten to `.html`, titles from the first heading, nav built live from `.order`, and `_config.yml`'s `favicon` and `wikidown.exclude_from_site` honored. Uses theme files from the wiki root when present (so customizations apply) and the built-in copy otherwise — running pages first is optional. `--base-url` prefixes theme links for sites served under a path (GitLab project sites); `--clean` empties the output folder first. This is the path for GitLab Pages, Azure Static Web Apps, any static host, and local preview — see [Publishing to GitHub Pages & Any other host](#).

See [Format](#) for the on-disk format the CLI maintains, including the relative-link convention that `check-links` enforces and the breadcrumb navigation it injects.

Editor

Wikidown ships a Blazor WebAssembly PWA that lets you browse and edit any Wikidown wiki straight from the browser. It's hosted on Azure Static Web Apps at `https://wikidown.app/` (default hostname `https://victorious-wave-03164381e.7.azurestaticapps.net/` still serves the editor, but OAuth sign-in only completes on `wikidown.app`), with a small Functions app at `/api/*` that only exists to complete OAuth token exchange — page commits still go directly to your provider's REST API from the browser.

Providers

- **GitHub** — Sign in with GitHub (OAuth) or paste a fine-grained Personal Access Token. Commits via the Contents API.
- **GitLab** — Personal Access Token with the `api` scope (Settings → Access tokens). Works with `gitlab.com` and self-managed instances (set the instance URL on the connect form). Reads via the Repository Files/Tree APIs, commits via the Commits API; conflict detection uses each file's `last_commit_id`. `gitlab.com` serves CORS-open API responses, so no proxy is involved.
- **Azure DevOps** — Personal Access Token. Commits via the Pushes API. (OAuth is planned but not yet wired.)

Access tokens are kept in browser `localStorage`. The Functions app never sees your repo contents and never stores your token — it only swaps an OAuth code for an access token and redirects back.

Sign-in flow (GitHub)

1. Click **Sign in with GitHub** on `/connect`.
2. Redirect to `github.com/login/oauth/authorize` for the Wikidown OAuth App.
3. Return to `/oauth/github/return.html` — a tiny static page served by ASWA. Its JS reads `code` + `state` from the query string and fetch-POSTs them as a JSON body to `/api/auth/github/exchange`.
4. The `GitHubExchange` Function swaps `code` + `client_secret` for an access token against `https://github.com/login/oauth/access_token` and returns `{"token":"..."}` (or `{"error":"..."}`).
5. The relay page redirects to `/connect#gh_token=...&gh_state=...`. The SPA validates the CSRF state, stores the token in `localStorage`, scrubs the hash, and navigates to `/browse`.

The two-piece relay exists because Azure Functions on ASWA managed backends treats `?code=` as a reserved function-key candidate and rejects requests that carry it with a pre-dispatch HTTP 500 — even on `AuthorizationLevel.Anonymous` routes. Since GitHub's OAuth redirect always appends `?code=...`, the callback URL has to land somewhere other than `/api/*`. See [Vibing Phase Recap](#) for the war story.

A "Use a PAT instead" toggle on the same page is kept as a fallback.

GitHub OAuth Apps support a single `Authorization callback URL`, so the Wikidown app is registered against `https://wikidown.app/oauth/github/return.html`. Sign-in on the ASWA default hostname will fail with a `redirect_uri` mismatch — use `wikidown.app` for OAuth, or paste a PAT as a fallback.

Browse and edit

- Read mode renders pages with MudMarkdown (CommonMark + KaTeX-ready).
- Edit mode is side-by-side: textarea on the left, live preview on the right.
- Save commits the change with a configurable message.

Conflict handling

The editor records the file SHA (or the branch-head commit OID, or on GitLab the file's `last_commit_id`, depending on the provider) before you edit. If the remote has moved when you save, the commit fails with a "Reload remote" banner so you don't clobber someone else's change.

Drafts

In-progress edits are kept per (provider, owner, project, repo, branch, page) in `localStorage`. They survive page reloads and crashes, and clear on a successful commit.

Agents

Wikidown ships drop-in configs so AI coding assistants can maintain your wiki by running the [wikidown CLI](#) without you wiring anything up by hand.

The CLI is the agent surface

There is no separate agent protocol: every config below tells the assistant to shell out to `wikidown -- list`, `read` (with `--section`), `write`, `edit`, `write-section`, `append`, `new`, `move`, `delete`, `reorder`, `search`, `walk`, plus `check-links` — instead of editing `/docs/*.md` directly, so `.order` files, breadcrumbs, and link targets stay consistent. Any host that can run a shell command (Claude Code, Copilot in VS Code or on github.com, the Copilot coding agent) gets the full read/write surface. Hosts that can't (Claude Desktop, a VS Code chat mode without terminal access) can't edit the wiki directly; the skill still carries the command list, so they can suggest the exact `wikidown` commands for you to run instead.

Shared skill

Claude and Copilot share one skill file in the Agent Skills standard format: [agents/skills/wikidown/SKILL.md](#). It carries the format rules, a `wikidown` command cheat sheet (which verb to reach for and when), the editing workflow, and a manual last-resort protocol for locked-down machines where the CLI can't run (no `.NET` plus an execution policy blocking the self-contained binary) — breadcrumb format, `.order` bookkeeping, and the move/delete invariants, so an agent can still edit correctly and flag the edits for later `wikidown check-links` verification. The same command list is what a shell-less host uses to suggest commands rather than run them.

- Claude Code loads it from `.claude/skills/wikidown/SKILL.md`.
- GitHub Copilot loads it from `.github/skills/wikidown/SKILL.md`.

Claude Code

In [agents/claude/](#), layered on top of the shared skill:

- `wikidown.subagent.md` → `.claude/agents/wikidown-editor.md` — a subagent definition that owns `/docs/*.md` reads and writes.
- `CLAUDE.md` snippet — append to your project's root `CLAUDE.md` so every Claude session knows to delegate wiki edits.

Copilot

In [agents/copilot/](#), layered on top of the shared skill:

- `copilot-instructions.md` → `.github/copilot-instructions.md` — repo-wide guidance.
- `wikidown.agent.md` → `.github/agents/wikidown.agent.md` — a custom agent, also used by the Copilot coding agent on github.com.
- `wikidown.chatmode.md` → `.github/chatmodes/wikidown.chatmode.md` — a custom chat mode focused on wiki editing. When the chat mode has no terminal it suggests the `wikidown` commands to run rather than running them.

Install

The recommended install is the [CLI](#):

```
dotnet tool install -g Wikidown.Cli
wikidown init --agents all
```

--agents accepts claude, copilot, all, or none. Existing files are skipped unless you pass --force; an existing CLAUDE.md gets the wiki section appended if it doesn't mention the wiki already.

Without .NET

The install scripts on wikidown.org fall back to a self-contained wikidown binary when dotnet isn't on PATH — nothing else to install:

```
curl -fsSL https://wikidown.org/install.sh | sh
wikidown init --agents all
```

```
irm https://wikidown.org/install.ps1 | iex
wikidown init --agents all
```

init scaffolds the same configs either way, and the agents run the same wikidown commands — the self-contained binary provides them (installed to ~/.wikidown/bin, or %USERPROFILE%\wikidown\bin on Windows). If a local execution policy blocks even that binary, the skill's last-resort protocol keeps wiki maintenance possible with plain file edits.

Manual copying still works: the [agents/README.md](#) maps each source file to its destination. Once installed, the agents run wikidown commands rather than editing files by hand, so .order files, breadcrumbs, and link targets stay consistent.

Keeping configs current

These files are copied once, not auto-updated. See [Updating](#) for how to re-run `wikidown init --agents all --force` to pick up config changes shipped in newer Wikidown releases.

Handy Prompts

Once your AI agent is wired up (see [Agents](#)), you don't need to remember wikidown CLI verbs or flags — just describe what you want in plain language. These are example prompts for the everyday wiki work that comes up after initial setup.

Adding a page

- "Add a new page called Troubleshooting under Getting Started, with a section on common install errors."
- "Create a page at /CLI/Advanced-Usage documenting the `--root` flag and the `WIKIDOWN_ROOT` environment variable."
- "I want a new top-level page called FAQ. Seed it with three placeholder questions I can fill in later."

Changing the content of a page

- "Update the Format page to add a note about how image paths resolve after a move."
- "Rewrite the intro paragraph on Home to mention the new `export -pdf` command."
- "Find every page that mentions Jekyll and add a short note that `export -html` doesn't need Ruby."
- "Read the CLI page back to me — I want to check the `check-links` section is still accurate before I edit it."

Reordering pages

- "Move Updating so it's the second page under Getting Started, right after Format."
- "Reorder the top-level nav so Agents comes before Editor."
- "List the pages under /Getting-Started in their current order."

Moving or renaming a page

- "Rename /Testing/Browser-Test-Plan to /Testing/Manual-Test-Plan and fix any links that point at it."
- "Move /Meta/Release-Quirks under /Getting-Started since it's really a setup gotcha."

Committing and pushing changes

- "Commit the wiki changes with a message summarizing what changed, then push."
- "Show me a diff of what you changed in /docs before committing."
- "Commit just the Handy Prompts page for now — I'm not ready to push the rest."

Other useful prompts

- "Search the wiki for every mention of 'breadcrumb' and summarize what's documented."
- "Run check-links and fix anything it flags."
- "Walk the whole wiki and tell me if any page looks out of date."

Release Notes

What changed in each Wikidown release, and where to get it. Newest first. For how to update an existing install, see [Updating](#).

0.10.0 — 29 September 2026

PDF exports are set in Atkinson Hyperlegible, a typeface designed by the Braille Institute for readers with low vision.

Where to get it

Component	Link
wikidown CLI (NuGet global tool)	Wikidown.Cli 0.10.0 — <code>dotnet tool update -g Wikidown.Cli</code>
Wikidown.Core library	Wikidown.Core 0.10.0
Self-contained CLI binaries (no .NET needed)	GitHub Release cli-v0.10.0 — win/linux/osx, x64 and arm64; or re-run the install script
Web editor	Updated in place; its Export PDF uses the same fonts

The Visual Studio extension is unchanged at 1.6.0.

What changed

- **New PDF fonts.** `wikidown export -pdf` and the web editor's PDF export set body text in Atkinson Hyperlegible Next and code in Atkinson Hyperlegible Mono (SIL Open Font License 1.1), both built into the tool as before.
- **Characters those fonts lack still print.** Arrows, box-drawing lines, Greek and Cyrillic fall back to the DejaVu fonts the export used before, character by character, instead of coming out blank.

0.9.0 and Visual Studio extension 1.6.0 — 27 September 2026

Kanban boards made of ordinary wiki pages, a `check-links` that passes on real wikis, and PDF exports that no longer drop content from lists. This CLI release also carries the Windows fixes that shipped in extension 1.5.4.

Where to get it

Component	Link
wikidown CLI (NuGet global tool)	Wikidown.Cli 0.9.0 — <code>dotnet tool update -g Wikidown.Cli</code>
Wikidown.Core library	Wikidown.Core 0.9.0
Self-contained CLI binaries (no .NET needed)	GitHub Release cli-v0.9.0 — win/linux/osx, x64 and arm64; or re-run the install script
Visual Studio extension	Wikidown Wiki Project on the Marketplace — or let Visual Studio auto-update
VSIX download	GitHub Release vsix-v1.6.0

What changed

- **Kanban.** `wikidown kanban init` creates a dashboard page with To Do, Doing and Done sub-pages; each item is a sub-page of one of them, and moving an item is an ordinary `wikidown move`. See [Kanban](#).
- **Headings written as links can be addressed by their text.** `--section "To Do"` and `--after "To Do"` find `## [To Do](...)`.
- **check-links ignores links inside code blocks and inline code.** Example links in documentation no longer count as broken, and no longer count as a parent page linking its children.
- **PDF export keeps everything in a list item.** A second paragraph, a code block, a quote or a table inside a list item used to be left out of the PDF without a word; it now appears under its bullet.
- **Windows fixes from extension 1.5.4** (line endings left alone, UTF-8 console, portable page names, case-only renames, `edit --delete`) now reach the NuGet tool and the standalone binaries.

Visual Studio extension 1.5.4 — 20 September 2026

A Windows-first pass. Most Wikidown users are on Windows, and a sweep of the code turned up several places that quietly assumed Unix. They are fixed in the extension and its bundled CLI now; the same fixes reach the NuGet tool and the standalone binaries with the next CLI release.

Where to get it

Component	Link
Visual Studio extension	Wikidown Wiki Project on the Marketplace — or let Visual Studio auto-update
VSIX download	GitHub Release vsix-v1.5.4

What changed

- **Line endings are left alone.** The CLI and the extension used to write LF into every file they touched — `.order`, new pages, whole-page rewrites, `init` and `pages` output — leaving a Windows checkout with mixed endings and a stream of git "LF will be replaced by CRLF" warnings. Now a rewritten file keeps the endings (and UTF-8 BOM) it already had, a new file matches the rest of the wiki, and a brand-new wiki uses the platform default. CRLF `.order` files were always read correctly; agents are now told so, and told never to "fix" them.
- **UTF-8 in and out.** The CLI sets the console to UTF-8, so `wikidown read > page.md` and `piped - -stdin` no longer mangle accents, dashes, and arrows on a Windows console, and edit summaries stop showing `?` where an en dash should be.
- **Page names that Windows can't store are refused** with a clear message — `Report: Q1` used to vanish into an NTFS alternate data stream, and `CON` echoed the page to the terminal while reporting success.
- **Case-only renames work:** `wikidown move --from guide --to Guide` no longer fails with "Destination exists" on Windows.
- **check-links is case-exact on every platform**, so a link that only resolves because Windows ignores case is caught before it 404s on GitHub Pages.
- **edit --delete** removes the matched text. The old spelling, `--new ""`, never reached the CLI from Windows PowerShell 5.1.
- **export-html --clean** empties the output folder instead of deleting it, which failed whenever Explorer or a terminal had it open.
- **Export to PDF from Visual Studio** quotes its arguments by Windows rules, fixing wiki roots that end in a backslash (a drive root).
- **Help, README, and agent instructions** show shell-neutral commands (`--file`, slash-less page paths, `.` for the root) and PowerShell alongside bash. CI now runs the test suite on Windows as well as Linux.

0.8.0 — 15 September 2026

The MCP server is gone: the `wikidown` CLI is now the only surface AI agents use. `Wikidown.Mcp` is no longer published, the `wikidown-mcp` executable is retired, and the `wiki_*` tools go with it.

Where to get it

Component	Link
wikidown CLI (NuGet global tool)	Wikidown.Cli 0.8.0 — <code>dotnet tool update -g Wikidown.Cli</code>
Wikidown.Core library	Wikidown.Core 0.8.0
Self-contained CLI binaries (no .NET needed)	GitHub Release cli-v0.8.0 — win/linux/osx, x64 and arm64; or re-run the install script
Source	markdov-is/Wikidown

The Visual Studio extension and the browser editor are unchanged in this release.

Why

Every `wiki_*` MCP tool was a thin wrapper over the same `wikidown` verb, so the CLI has had full parity since 0.7.0. Shipping two surfaces meant two things to keep correct, two things to document, and two version numbers that had to move together. One surface is easier to keep correct, and an agent that can run a shell loses nothing.

What to do

1. Uninstall the old server: `dotnet tool uninstall -g Wikidown.Mcp`.
2. Delete `.mcp.json` and `.vscode/mcp.json` from downstream repos — they only wired up `wikidown-mcp`.
3. Re-run `wikidown init --agents all --force` to refresh the agent configs. The skill, the Claude Code subagent, and the Copilot agent and chat mode now instruct agents to run the CLI — see [Agents](#).

The tool names map one-to-one onto CLI verbs, so prompts, notes, or scripts that mention them translate directly:

MCP tool	CLI verb
<code>wiki_list</code>	<code>wikidown list</code>
<code>wiki_read</code> (with section)	<code>wikidown read</code> (with <code>--section</code>)
<code>wiki_write</code>	<code>wikidown write</code>
<code>wiki_edit</code>	<code>wikidown edit</code>
<code>wiki_write_section</code>	<code>wikidown write-section</code>
<code>wiki_append</code>	<code>wikidown append</code>
<code>wiki_new</code>	<code>wikidown new</code>
<code>wiki_move</code>	<code>wikidown move</code>
<code>wiki_delete</code>	<code>wikidown delete</code>
<code>wiki_reorder</code>	<code>wikidown reorder</code>
<code>wiki_search</code>	<code>wikidown search</code>
<code>wiki_walk</code>	<code>wikidown walk</code>

Chat-only hosts that can't run a shell (Claude Desktop, a VS Code chat mode without a terminal) can no longer edit a wiki directly. The skill gives them the command list, so they can suggest the exact wikidown commands for you to run instead.

0.7.0 — 12 September 2026

The small-change toolkit for agents. Until now the only way to change a page through the MCP server or CLI was to rewrite all of it; most of a long editing session on a large wiki was whole-page round-trips for one-line changes. This release adds four patching tools so an agent pays only for the text it changes.

Where to get it

Component	Link
wikidown CLI (NuGet global tool)	Wikidown.Cli 0.7.0 — dotnet tool update -g Wikidown.Cli
wikidown-mcp MCP server (NuGet global tool)	Wikidown.Mcp 0.7.0 — dotnet tool update -g Wikidown.Mcp
Wikidown.Core library	Wikidown.Core 0.7.0
Self-contained CLI binaries (no .NET needed)	GitHub Release cli-v0.7.0 — win/linux/osx, x64 and arm64; or re-run the install script
Source	markdov-is/Wikidown

The Visual Studio extension and the browser editor are unchanged in this release.

New: patching tools

All four are in the MCP server and the CLI. They match the raw markdown exactly, detect and preserve the file's line endings (CRLF or LF), never touch the breadcrumb line or .order, never create a page, and fail with a message that says what to do next — the page's headings on a section miss, the match count on an ambiguous edit. See [CLI](#) § Commands (the MCP Server page was retired with 0.8.0).

- **wiki_edit / wikidown edit** — replace an exact substring in place, with the same contract as Claude Code's Edit tool: old, new, optional replaceAll. Returns the changed line numbers with two lines of context so the agent needn't re-read the page.
- **wiki_read_with_section / wikidown read --section** — return one heading plus everything under it up to the next heading of the same or higher level, so a ## section includes its ### children. Heading match is case-insensitive and ignores #s and whitespace; a miss lists every heading on the page.
- **wiki_write_section / wikidown write-section** — replace the body under one heading, keeping the heading line and the rest of the page. ### children inside the section are replaced too. Exactly one blank line is kept around the new body, so repeated writes are idempotent. createIfMissing appends a new ## section instead of failing.
- **wiki_append / wikidown append** — add a block at the end of a page, or at the end of one section with afterSection, always separated by exactly one blank line. Repeated appends never stack blank lines.

Agent guidance

The shared skill, the Claude Code subagent, the Copilot agent and chat mode, and the CLAUDE.md snippet now say: prefer wiki_edit for small changes, wiki_write_section for one section, wiki_append to add at the end, and wiki_write only for new pages or full rewrites. Re-run wikidown init --agents all --force in a downstream repo to pick these up — see [Agents](#).

Fixes

- **MCP errors now say what went wrong.** The MCP SDK reported every failure as a bare `An error occurred invoking 'wiki_x'`, which hid messages such as `page already exists` from `wiki_new`. Every tool now returns its real message.
- **Concurrent tool calls no longer collide.** Hosts such as Claude Code batch independent tool calls, and two edits to the same page in one batch raced on the file. Tool calls are now serialized.
- **wikidown `--help` / `-h`** no longer demands a value or valid command arguments.

Upgrading

Follow [Updating](#). One gotcha specific to the MCP server: `dotnet tool update -g Wikidown.Mcp` fails with `Access to the path ... is denied` while any Claude Code, Claude Desktop, or VS Code session has `wikidown-mcp` running. Close those sessions (or stop every `wikidown-mcp` process) first, update, then reopen them. To confirm you are on the new version, `dotnet tool list -g` must show `0.7.0` for both tools, `wikidown edit --help` must print a usage block, and the MCP tool list must show 12 tools including `wiki_edit`.

Pull requests

[#23](#) `wiki_edit` · [#24](#) `wiki_read(section)` · [#25](#) `wiki_write_section` · [#26](#) `wiki_append` · [#27](#) landing + version bump. Issues [#19](#), [#20](#), [#21](#), [#22](#).

0.6.0 — 2 September 2026

- `wikidown export-html`: render the wiki to a static site with the starter theme in .NET (Markdig + Fluid), no Jekyll or Ruby — the path for GitLab Pages, Azure Static Web Apps, and local preview.
- `wikidown.org` merged into `/docs`: the product site is now this wiki, published with `export-html`.
- Editor: GitLab provider.
- Fixes: pages and `export-html` nav linked `/Home` to `Home.html` instead of the site root.

Binaries: [cli-v0.6.0](#).

Testing

Test plans and QA checklists for Wikidown's public surfaces.

Pages

- [Browser Test Plan](#) — end-to-end manual or automated browser checks across the marketing site, editor PWA, and custom domain. Designed to be handed to an AI agent with a browser-automation tool (e.g. Playwright MCP).

Browser Test Plan

A handoff checklist for an AI agent (or human) driving a real browser — e.g. via Playwright MCP — to exhaustively smoke-test Wikidown's public surfaces.

How to use this page

Wikidown now has two distinct public surfaces:

1. **Marketing site** — `https://wikidown.org/` (static HTML + CSS, served by GitHub Pages; marketing only, no editor, no API).
2. **Editor PWA + API** — `https://wikidown.app/` (Blazor WASM + MudBlazor at `/`, .NET isolated Functions at `/api/*`, served by Azure Static Web Apps). The ASWA default hostname `https://victorious-wave-03164381e.7.azurestaticapps.net/` is still bound and should behave identically.

For every check, report **PASS**, **FAIL**, or **SKIPPED** with a one-line reason. On any failure, attach:

- URL at time of failure
- Viewport size
- Browser + version
- Console errors (if any)
- Network errors (if any)
- Screenshot

Checks are ordered so you can stop early on a catastrophic failure (e.g. site doesn't load).

Marketing site — `https://wikidown.org/`

Load and assets

- GET `/` returns HTTP 200 and renders without console errors.
- `<title>` is set and non-empty; favicon visible in the tab.
- Logo image loads and has **transparent corners** — no white square behind the rounded glyph on the page background. Sample pixels at the four corners of the logo element to confirm alpha = 0.
- All `<img>` / `<link rel="stylesheet">` resources return 200 (Network panel has no reds).
- No mixed-content warnings; every asset is HTTPS.
- `og:title`, `og:description`, and `og:image` meta tags are present.

Navigation

- **"Open the editor"** button → `https://wikidown.app/`.
- **"View on GitHub"** has `target="_blank"` and resolves to HTTP 200.
- Every nav / hero / footer link resolves to HTTP 200 — none are `#` or `javascript:void(0)`.
- GET `/app/` on the marketing origin returns 404 — the editor is **not** served from the marketing site anymore.

Layout and readability

- The feature-card grids are fully visible — no horizontal overflow, no text clipped, no card overlaps another.
- Each card's kicker, heading, and body text are legible.
- The "Built for humans and agents" row has three equal-width cards on desktop with aligned code blocks at the bottom.

Mobile breakpoint

- At viewport width ~480 px, grids stack into a single column without horizontal scroll.
- Hero stacks (logo above or below the copy; not overlapping).
- Hero text stays readable (no character clipped, no font <14 px).
- `document.documentElement.scrollWidth == document.documentElement.clientWidth`.

Editor PWA — <https://wikidown.app/>

Shell and first paint

- GET / returns HTTP 200 and the Blazor WASM app reaches interactive state (loading spinner disappears within ~10 s on a fast connection).
- App bar shows the **W↓ logo** immediately to the left of the word **Wikidown**.
- The **Drafts** button is visible in the app bar. Clicking it on a clean profile shows the empty state (no drafts).
- Home page content renders — no perma-spinner, no red error banner.
- Browser console shows no unhandled exceptions during load.

Static assets

- GET /favicon.ico → 200.
- GET /apple-touch-icon.png → 200.
- GET /manifest.webmanifest → 200. Content-Type is application/manifest+json (or application/json). Body is valid JSON and declares name, short_name, start_url, display, and at least one icons entry.

Service worker

- DevTools → Application → Service Workers lists a registered SW for scope /, status **activated and is running**.
- DevTools → Application → Cache Storage shows at least one offline-cache-* entry populated after a reload.
- A hard reload (Ctrl+Shift+R) still produces a working app; a subsequent normal reload is served partly from cache (Network panel shows some requests flagged (ServiceWorker) or (disk cache)).

API smoke tests — /api/*

The /api/* routes are served by the linked Azure Functions app. These are quick unauthenticated probes — they don't touch real OAuth.

- GET /api/config/github → HTTP 200, Content-Type: application/json, body shape {"clientId": "..."} . If clientId is empty, the SWA's GITHUB_CLIENT_ID application setting isn't configured yet — mark **FAIL** and stop the OAuth section below.
- GET /api/config/ado → HTTP 200, body shape {"clientId": "..."} (empty string acceptable — ADO OAuth is not wired yet).
- POST /api/auth/github/exchange with body {"code": "not-a-real-code", "state": "xyz"} and Content-Type: application/json → HTTP 502 with body {"error": "token_exchange_failed"} (GitHub returns non-200 for the bad code). Confirms the token-exchange round-trip is wired.

- POST `/api/auth/github/exchange` with body `{}` → HTTP 400 with body `{"error": "missing_code"}`.
- GET `/oauth/github/return.html` → HTTP 200, serves the static relay page (briefly shows "Completing GitHub sign-in..." before redirecting to `/connect#gh_error=missing_code` because no code is in the query string).
- **Do not** add `?code=...` to any `/api/*` URL when probing — Azure Functions on ASWA's managed backend rejects `?code=` queries pre-dispatch with HTTP 500, regardless of `AuthorizationLevel`. The relay-via-static-page flow exists specifically to avoid this. See `/Meta/Vibing-Phase-Recap`.

GitHub OAuth sign-in flow (*skip by default*)

Only run if a throwaway GitHub account is available to approve the Wikidown OAuth App. Without one, mark every check in this section **SKIPPED**.

With throwaway credentials:

- Navigate to `/connect`. The GitHub tab shows a **Sign in with GitHub** button as the primary action, *not* a PAT field by default. A "Use a PAT instead" toggle is visible.
- Fill Owner + Repo for a repo the throwaway account can access.
- Click **Sign in with GitHub**. Browser leaves the app and lands on `github.com/login/oauth/authorize?...` with the Wikidown app name visible on the consent screen.
- Approve. Browser returns through `/oauth/github/return.html?code=...&state=...`, briefly shows "Completing GitHub sign-in..." while the relay page POSTs the code to `/api/auth/github/exchange`, then lands on `/connect` with the hash immediately cleaned up by `history.replaceState` (inspect the address bar — no `#gh_token=` left behind after first paint).
- The app auto-navigates to `/browse` and the snackbar reads "Connected to <owner>/<repo>".
- DevTools → Application → Local Storage has key `wikidown.connection.v1` with a JSON value that includes `token` (the access token) and `provider: "GitHub"`.
- DevTools → Application → Session Storage is empty — `wikidown.gh_state` and `wikidown.gh_pending` have been cleared.
- Returning to `/connect` shows the "Connected: ..." banner with **Browse** and **Disconnect** buttons.
- Clicking **Disconnect** removes `wikidown.connection.v1` from Local Storage and the connected banner disappears.

State-mismatch negative test:

- Start a sign-in, then manually set `sessionStorage['wikidown.gh_state']` to a different value before returning from GitHub. The `/connect` page should show a "state mismatch" error and **not** store a connection.

ASWA default hostname — `https://victorious-wave-03164381e.7.azurestaticapps.net/`

The vanity domain is `wikidown.app`, but the ASWA default hostname stays bound as a safety net. Every check above should behave identically here **except GitHub OAuth sign-in**, which only completes on `wikidown.app`.

- GET `/` returns 200 and reaches interactive state.
- TLS certificate is valid (Azure-managed, no browser interstitial).
- `/api/config/github` and `/api/ping` respond the same as on `wikidown.app`.
- **GitHub OAuth sign-in is expected to FAIL here.** GitHub OAuth Apps support a single `Authorization callback URL`, which is registered against `https://wikidown.app/oauth/github/return.html`. Starting sign-in from the ASWA default hostname should produce a `redirect_uri mismatch` error from `github.com/login/oauth/authorize`. The PAT fallback still works here.

PWA install

Run in a Chromium-based browser (Chrome or Edge) where install prompts are supported.

- At the editor origin, the install affordance appears — either a native install prompt fires, or the install icon shows in the address bar.
- Triggering install opens a standalone app window.
- The installed PWA's taskbar / dock / launchpad icon is the Wikidown `W↓` icon, not a generic globe.
- Launching the installed PWA opens without browser chrome and lands on the editor Home page.
- Uninstalling cleanly removes the icon and leaves no zombie window.

Responsive and theme

Viewport widths

Test both live surfaces (marketing + editor) at:

- **360 px** (small phone) — no horizontal scroll, tap targets ≥ 40 px, all text readable.
- **768 px** (tablet) — layout adjusts, no awkward large gaps, nav still usable.
- **1280 px** (laptop) — full desktop layout, no stretched hero image, content max-width looks intentional.

Landscape on mobile

- At 812×375 (iPhone landscape), the editor app bar stays visible and consumes $\leq \sim 15\%$ of vertical space.
- Marketing hero does not push the CTA below the fold unreachably.

Theme

- Toggle OS-level dark mode. The marketing site and editor either follow it or stay in their chosen theme — but never produce unreadable dark-on-dark or light-on-light text.
- No flash of wrong theme (FOUC) on reload.

Regression checklist

A 10-item fast pass for any future deploy:

1. `https://wikidown.org/` returns 200 and renders.
2. Logo has transparent corners on the marketing site.
3. "Open the editor" button reaches `https://wikidown.app/`.
4. The editor origin returns 200 and reaches interactive state with no console errors.
5. App bar shows `W↓ Wikidown` with Drafts menu visible.
6. `manifest.webmanifest` returns 200 and is valid JSON.
7. Service worker is registered and activated.
8. `GET /api/config/github` returns 200 with a non-empty `clientId`.
9. `GET /oauth/github/return.html` returns 200 (static relay page) and `POST /api/auth/github/exchange` with `{}` returns 400 `{"error": "missing_code"}`.
10. 480 px mobile layout stacks without horizontal scroll.

Known gaps

- **GitHub OAuth flow requires throwaway credentials** and an approved consent on the Wikidown OAuth App — SKIPPED without them.
- **ADO OAuth is not wired yet.** `/api/config/ado` returns an empty `clientId` on purpose; ADO tab still uses PAT.

- **No device-flow test.** GitHub Device Flow isn't implemented yet; it's only useful for CLI / MCP, not for this browser test.
- **No accessibility audit here.** A11y (axe, Lighthouse a11y) should live in a dedicated page.
- **No performance budgets.** Lighthouse scores are informational; thresholds aren't defined yet.

Reporting format

When you finish, produce a short summary like:

```
Marketing site: 12/12 PASS
Editor PWA shell: 9/9 PASS
API smoke tests: 4/4 PASS
GitHub OAuth: SKIPPED (no throwaway creds)
Custom domain: SKIPPED (not live)
PWA install: 4/5 PASS (1 failure, see screenshot)
Responsive/theme: 7/7 PASS
Regression (10): 10/10 PASS
```

Attach every failure screenshot and a combined HAR of both surfaces.

Meta

Behind-the-scenes notes about how Wikidown itself gets built — build-log posts, phase recaps, architectural decisions that didn't fit in the main docs.

Pages

- [Vibing Phase Recap](#) — build log from getting the WASM editor, Functions API, marketing site, and CLI/MCP tool pair standing up.
- [Release Quirks](#) — maintainer-facing gotchas in this repo's NuGet and VS Marketplace VSIX release pipelines, discovered the hard way.

Wikidown — "Vibing Phase" Recap

A build log / field notes of getting the basics standing up, from nothing to a deployed WASM editor with a working Functions API, a marketing site on its own domain, and a CLI + MCP tool pair.

The shape of the thing

Wikidown is a structured markdown wiki that lives in `/docs` of any git repo — no service to run, tokens never leave the browser. One codebase ships four surfaces:

- **Wikidown.Core** — the page model: `.order` files, title-form links (`/Parent/Child`), on-disk read/write.
- **Wikidown.Cli** — a `dotnet tool` called `wikidown` for `list / read / write / move / search`.
- **Wikidown.Mcp** — an MCP stdio server exposing the same surface to Claude Code, Claude Desktop, and VS Code Copilot.
- **Wikidown.Web** — a Blazor WASM + MudBlazor PWA editor.
- **Wikidown.Site** — a static marketing page.
- **Wikidown.Api** — a .NET isolated Azure Functions app for the OAuth token exchange.

The wiki eats its own dogfood: this repo's own `/docs` is a Wikidown wiki, edited through the `wikidown-editor` subagent and the `wikidown` skill.

Deployment topology we landed on

After one wrong turn (more below), we split the surfaces cleanly:

- **wikidown.org** — GitHub Pages, marketing only. Plain HTML/CSS, zero backend, Porkbun DNS with four A records to GitHub's 185.199.108-111.153.
- **wikidown.app** — Azure Static Web Apps. Blazor WASM editor at `/`, managed Functions at `/api/*`. The ASWA default hostname `victorious-wave-03164381e.7.azurestaticapps.net` stays bound as a safety net.

GitHub Pages is the wrong host for OAuth (no runtime), so anything that needs a server — token exchange, config endpoints — lives on ASWA. Marketing-only stays on Pages because it's free, cached globally, and wiring a .NET WASM app into it gets you nothing.

GitHub OAuth, the real way

The first cut of `/connect` was PAT-only with a frozen "GitHub OAuth Device Flow needs a small proxy..." caption. That caption was still true — a proxy was needed — so we built one.

- **/api/config/github** — returns the public `clientId` (empty string means the SWA app setting isn't wired).
- **/oauth/github/return.html + /api/auth/github/exchange** — a static HTML relay page is the registered OAuth callback. Its JS reads `code + state` from the query string and POSTs them as a JSON body to the `GitHubExchange` Function, which swaps `code` for a token against `https://github.com/login/oauth/access_token` using the server-side `GITHUB_CLIENT_SECRET` and returns `{ "token": "..." }`. The page then redirects to `/connect#gh_token=...&gh_state=...`. The reason for the relay (rather than a single Function callback) is documented in §6 below.
- **/connect in the WASM app** — generates a CSRF `state`, stashes pending form + `state` in `sessionStorage`, redirects to `github.com/login/oauth/authorize`, receives the fragment on return, validates `state`, stores the access token in `localStorage`, and scrubs the fragment via `history.replaceState` so no token is ever left in the address bar. A "Use a PAT instead" toggle keeps the old path available.

Device Flow was consciously deferred. It's the right answer for the CLI / MCP surfaces, not for a browser.

Things that went sideways, and what they taught us

1. .NET 10 vs the world

The repo is on .NET 10 (pinned 10.0.100 in `global.json`) because it's fun and we can. Azure's Oryx builder — the image that powers `Azure/static-web-apps-deploy` — doesn't have a .NET 10 SDK yet. First ASWA run exploded with *"Requested SDK version 10.0.100 ... Installed SDKs: [none match]"*.

Pattern that worked:

- **Pre-build the Blazor WASM editor on the GitHub runner** where `actions/setup-dotnet@v4` can install any SDK, then hand the published `wwwroot` to the deploy action with `skip_app_build: true`.
- **Let Oryx build the Functions API**, but drop a scoped `src/Wikidown.Api/global.json` pinning 9.0.x with `rollForward: "latestMinor"` so Oryx picks its bundled 9.0 SDK instead of tripping on the root 10.0 pin. Letting Oryx do the API build also means it writes the `functions.metadata` file the deploy step needs to register functions.

Lesson: when you're ahead of the hosted build image, do the build on the runner and make the deploy action a pure uploader.

2. Functions language version

Oryx's first API-side error was unusually helpful: *"dotnetisolated versions are valid: 8.0, 9.0"*. We'd targeted `net10`. Retargeted `Wikidown.Api` alone to `net9`, everything else stayed on `net10`. Production `net9` artifacts on a `net9` isolated worker, no drift.

We also briefly had the API `global.json` on `latestMajor` — Oryx dutifully downloaded SDK 10.0.200 to build a `net9` project. That's the drift we didn't want; tightening to `latestMinor` kept SDK and TFM aligned.

3. The service worker ate our API

Most satisfying bug of the phase. After deploy, every API function registered cleanly — the portal showed all five. But hitting `/api/ping` in a browser tab loaded the Blazor shell and rendered Blazor's own "Not Found" page. The `staticwebapp.config.json` correctly excluded `/api/*` from navigation fallback, so SWA was blameless.

The Blazor PWA service worker — the one template ships with — intercepts `mode === 'navigate'` fetches, finds the URL isn't in the asset manifest, and cheerfully serves cached `index.html`. The request never left the browser. Our `/api/*` route excludes were configured on the wrong layer.

Fix was four lines: in `service-worker.published.js`, detect same-origin `/api/*` and return from the fetch handler *before* calling `event.respondWith`, so the browser makes a plain network request SWA can route.

Lesson: in a Blazor PWA + co-located API setup, the service worker is the *first* router the request hits. Its exclude list is more important than any `staticwebapp.config.json` rule, because SWA never even sees the request if the SW shortcuts it.

4. The Porkbun ALIAS detour

`wikidown.org` briefly broke because Porkbun was serving an ALIAS → `markdov-is.github.io` record. GitHub Pages apex domains want four plain A records (185.199.108.153 / .109.153 / .110.153 / .111.153). Swapping them in made `NotServedByPagesError` go away and Let's Encrypt issued the cert within ~15 minutes.

5. The "(managed)" red herring

Azure Portal's SWA → APIs blade shows a blue banner: *"Bring your own API backends are not supported in the Free hosting plan. Click to upgrade."* Right below it, Production → Function App → (managed). Those two things look contradictory but aren't — the banner only applies to BYO backends (external Function App, Container App, API Management). The managed Functions backend that ships with Free tier is right there, working, no upgrade needed. Worth calling out because the framing is actively misleading.

6. Azure Functions reserves ?code=

The "OAuth works locally, returns 500 on production" classic, but with a twist. Symptoms:

- Click **Sign in with GitHub** on `wikidown.app/connect`. GitHub authorizes fine and 302s the browser to `https://wikidown.app/api/auth/github/callback?code=<real-code>&state=...`
- Browser lands on a generic `wikidown.app` can't currently handle this request — HTTP ERROR 500. URL stays on the callback path; no redirect to `/connect#gh_error=...` despite a try/catch in the callback that *always* returns a 302.
- Application Insights records nothing. The portal toggle for AI keeps reverting to off.
- The GitHub OAuth App's client secret shows **Never used**, 18+ hours after creation, despite repeated sign-in attempts.

Hours of bisecting added canary endpoints — `/api/version`, `/api/ping`, an outbound-HTTP probe at `/api/github-test` — and verified each pass directly. The callback function with no code param (GET `/api/auth/github/callback`) correctly redirected to `/connect#gh_error=missing_code`. Adding `?code=anything` flipped it to a 500.

The smoking-gun test was one URL in a regular browser tab:

`https://wikidown.app/api/ping?code=foo` → 500. Ping is a one-line Function with `AuthorizationLevel.Anonymous`, no DI beyond logging, no outbound calls. The only thing that changed was the query string.

Azure Functions' HTTP host treats the query parameter ?code= as a function-key candidate and rejects unmatched keys with HTTP 500 before dispatching the function — even on AuthorizationLevel.Anonymous routes. GitHub OAuth's redirect ALWAYS appends `?code=...`. So a GitHub OAuth callback URL on a managed Functions endpoint is structurally impossible.

Fix: stop letting `code=` reach the Functions host as a query param.

- Register the OAuth callback URL against a static HTML page served by ASWA, not by Functions. We chose `/oauth/github/return.html`.
- The page's JS reads `code` and `state` from `window.location.search`, then fetch-POSTs them as a JSON body to a new `GitHubExchange` Function at `/api/auth/github/exchange`. The POST URL contains no `code=`, so the host leaves it alone.
- `GitHubExchange` does the `https://github.com/login/oauth/access_token` exchange and returns `{"token": "..."} or {"error": "...}"`.
- The relay page redirects to `/connect#gh_token=...&gh_state=...` (or `...#gh_error=...`), preserving the existing fragment convention so `Connect.razor`'s return-handling code is unchanged.
- The PWA service worker now also early-returns on `/oauth/*` (in addition to `/api/*`) so the relay page always hits the network.

Lessons:

- **AuthorizationLevel.Anonymous does not actually disable the ?code= key check on ASWA managed Functions** — at least not on the version we deployed against. The host inspects the query before checking the function's auth level.
- When App Insights toggles itself off and your function never appears in the logs, the host is rejecting the request *pre-dispatch*. The function never ran, so there's nothing to log.
- "Never used" on the OAuth App's client secret, after a day of real attempts, is a stronger signal than any 500 page. If GitHub never recorded a token-exchange POST hitting it, your callback isn't reaching the network — go upstream from the function code.

- One-URL bisection beats theory: `?param=foo` on a known-working endpoint is the cheapest possible way to falsify "the host doesn't touch query params."

Working state as of the handoff

- `https://wikidown.org/` — marketing site, live, HTTPS.
- `https://wikidown.app/` — editor PWA, live on the custom domain, OAuth-aware Connect page shipping. The ASWA default hostname `https://victorious-wave-03164381e.7.azurestaticapps.net/` is still bound as a safety net.
- All 7 Functions registered on the managed backend: `Ping`, `Version`, `GitHubProbe`, `GitHubConfig`, `AdoConfig`, `GitHubExchange`, `AdoCallback`.
- `ci.yml` builds + tests + packs on every PR; `pages.yml` deploys the marketing site; `azure-static-web-apps-*.yml` pre-builds the editor on the runner and hands it + the API source to Oryx.
- Browser test plan in `/docs/Testing/Browser-Test-Plan.md` covers the two surfaces, the API smoke tests, and the OAuth flow.

Known loose ends going into the inner loop

- Stale service worker needs evicting on the dev's machine before `/api/ping` is reachable from their regular browser.
- `GITHUB_CLIENT_ID` / `GITHUB_CLIENT_SECRET` need to be confirmed set on the SWA's Configuration → Application settings.
- The Wikidown GitHub OAuth App is registered with a single `Authorization callback URL`, `https://wikidown.app/oauth/github/return.html` — OAuth sign-in on the ASWA default hostname will fail with a `redirect_uri` mismatch by design. Converting to a GitHub App would allow multiple callback URLs; not worth it just for the fallback.
- ADO OAuth is stubbed — `/api/config/ado` returns empty `clientId` on purpose; ADO tab still uses PAT.
- Device flow (for CLI / MCP) is deferred until there's a real reason to ship it.

The vibe, honestly

This phase was a lot of "the docs say X; the platform does Y; the build runs but the thing 404s anyway." The pattern that paid off over and over: when the hosted builder can't do it, move the work onto the GitHub runner. When the service worker is in the way, read the service worker. When the Azure Portal banner contradicts the row below it, read the row below it.

What's in place is the boring-but-load-bearing foundation — infra, auth, deploy, two live origins, a dogfed wiki, a CLI, an MCP server, a PWA. Next loop gets to actually build features on top.

See also

- [Release Quirks](#) — maintainer-facing gotchas in the NuGet and VS Marketplace VSIX release pipelines, in the same "learned it the hard way" spirit as this page.

Release Quirks

Maintainer-facing notes on this repo's two release pipelines — NuGet packages and the VS Marketplace VSIX — covering things that aren't obvious from reading the workflow YAML alone. For the consumer-facing side of the same pipelines (how a downstream repo picks up a release once it ships), see [Updating](#).

NuGet (Wikidown.Core / Wikidown.Cli)

Driven by [release.yml](#).

- **Trigger is a path filter, and it's narrower than the test matrix.** The workflow only fires on a push to main touching `Directory.Build.props`, `src/Wikidown.Cli/**`, `src/Wikidown.Core/**`, `assets/**`, or the workflow file itself. It does **not** include `src/Wikidown.Pdf/**` — even though the workflow's own `dotnet test Wikidown.sln` step exercises `Wikidown.Pdf` code, because `Wikidown.Cli` references it. A `Wikidown.Pdf`-only change won't auto-trigger a release, even when it plausibly should. Trigger one manually instead:

```
gh workflow run release.yml --ref main
```

(`workflow_dispatch` is enabled for exactly this case.)

- **Version comes from `<VersionPrefix>` in `Directory.Build.props`, and nothing bumps it for you.** The publish step uses `--skip-duplicate`, so pushing without bumping `VersionPrefix` doesn't fail — it silently no-ops: the packages get repacked at the same version, `dotnet nuget push` tries to publish, NuGet.org rejects it as a duplicate, and the step reports success anyway. A green workflow run is not proof a new version shipped; check whether `VersionPrefix` actually moved in the commit that triggered it.
- **NuGet.org indexing lags behind a green run.** After the workflow finishes, `https://api.nuget.org/v3-flatcontainer/<package-id-lowercase>/index.json` can take several minutes to reflect the new version. Don't treat "not there yet" a minute after a green run as a sign something failed — give it time before investigating further.

VS Marketplace VSIX (Wikidown.Vs)

Driven by [vsix.yml](#).

- **Its version number is completely separate from the NuGet packages.** It comes from `<IdentityVersion="...">` in `src/Wikidown.Vs/source.extension.vsixmanifest`. Bumping `Directory.Build.props`'s `VersionPrefix` has zero effect here — these are two unrelated semver numbers that happen to look alike.
- **A plain push to main never publishes anything, even one that changes `Wikidown.Vs` source.** The workflow triggers on every push to main touching `src/Wikidown.Vs/**`, `Directory.Build.props`, or the workflow file itself, but that only runs the build + verify steps as a CI check. The two steps that actually ship a release — "Publish to Visual Studio Marketplace" and "Attach VSIX to GitHub Release" — are both gated on `if: startsWith(github.ref, 'refs/tags/vsix-v')`. Shipping a new VSIX version is a two-part action:

1. Bump the manifest version, commit, push to main.
2. Separately create and push a matching tag:

```
git tag vsix-v1.5.0
git push origin vsix-v1.5.0
```

Only the tag push triggers the Marketplace publish and the GitHub Release attach — pushing the version-bump commit alone does not.

- **Publisher identity** lives in `src/Wikidown.Vs/publish.json` (publisher `MarkDavis`, internal name `wikidown`) — this is what `VsixPublisher.exe` reads when it authenticates and uploads.
- **Requires the VSIX_PAT repo secret** (a Marketplace publish PAT). It's an Azure DevOps PAT (scope: All accessible organizations, Marketplace → Manage), created 2026-07-30 and expiring around late July

2027 — once it expires, publish runs start failing with auth errors. Renew by generating a fresh token at `aex.dev.azure.com` and running `gh secret set VSIX_PAT`.

- **The bundled-CLI build step needs its own restore, and leaks stray output folders.**

`Wikidown.Vs.csproj` has a `PublishBundledCli` MSBuild target that bundles a framework-dependent copy of `Wikidown.Cli` into the VSIX (under `Tools\cli\`, used by the extension's "Export to PDF" command to shell out via `dotnet exec`). At run time the extension prefers a globally installed `wikidown` tool and only uses the bundled copy as a fallback, so a stale bundle only affects users without the global tool. That target's nested `<MSBuild Projects= "...Wikidown.Cli.csproj" Targets="Restore;Publish">` call needs its own explicit `Restore` — `vsix.yml`'s "Restore NuGet packages" step only restores `Wikidown.Vs.csproj` itself, not `Wikidown.Cli`, so a plain `Targets="Publish"` fails on a clean CI checkout with `project.assets.json` not found. Separately: the outer `msbuild` invocation in `vsix.yml`'s "Build VSIX" step passes `/p:OutDir=...`, and that property leaks into the nested `Wikidown.Cli` / `Wikidown.Core` / `Wikidown.Pdf` builds too, leaving stray `artifacts/vsix/` folders in their own project directories. Harmless — it doesn't affect what ships in the VSIX — but confusing if you reproduce the exact CI command locally and go looking for where those folders came from.

See also

- [Updating](#) — how a downstream repo picks up new releases from both pipelines.
- [Vibing Phase Recap](#) — build-log notes on getting the rest of Wikidown's infra (WASM editor, Functions API, marketing site) standing up.